

Timed Automata

Sayan Mitra

Verifying cyberphysical systems

mitras@illinois.edu

Outline

- Review: Modeling cyber-physical systems
 - Hybrid automata
 - Executions
 - Urgency
 - Zeno
 - Hybrid stability

Hybrid Automaton

$$\mathcal{A} = (X, \Theta, A, \mathcal{D}, \mathcal{T})$$

- X : set of **state variables**
 - $Q \subseteq \text{val}(X)$ set of **states**
- $\Theta \subseteq Q$ set of **start states**
- set of **actions**, $A = E \cup H$
- $\mathcal{D} \subseteq Q \times A \times Q$
- $\mathcal{T}$: set of **trajectories** for X which is closed under **prefix, suffix, and concatenation**

Special Classes of Hybrid Automata

Algorithmic analysis of (Alur-Dill's) Timed Automata[1]

Timed Automata are a restricted class of hybrid automata with only clock variables which are amenable to automatic verification

[1] Rajeev Alur and David L. Dill. [A theory of timed automata](#). Theoretical Computer Science, 126:183-235, 1994.

Clocks and Clock Constraints

- A **clock variable** x is a continuous (analog) variable of type real such that along any trajectory τ of x , for all $t \in \tau.\text{dom}$, $(\tau \downarrow x)(t) = t$.
- For a set X of clock variables, the set $\Phi(X)$ of **integral clock constraints** are expressions defined by the syntax:

$$g ::= x \leq q \mid x \geq q \mid \neg g \mid g_1 \wedge g_2$$

where $x \in X$ and $q \in \mathbb{Z}$

- Examples: $x = 10$; $x \in [2, 5)$; true are valid clock constraints
- What do clock constraints look like geometrically in state space?
- Semantics of clock constraints $[g]$

Integral Timed Automata

Definition. An **integral timed automaton** is a HIOA $\mathcal{A} = \langle V, \Theta, A, \mathcal{D}, \mathcal{T} \rangle$ where

- $V = X \cup \{l\}$, where X is a set of n clocks and l is a discrete state variable of finite type L
- A is a finite set
- $\mathcal{D}$ is a set of transitions such that
 - The guards are described by clock constraints $\Phi(X)$
 - $\langle x, l \rangle - a \rightarrow \langle x', l' \rangle$ implies either $x' = x$ or $x = 0$
- $\mathcal{T}$ set of clock trajectories for the clock variables in X

Example: Light switch

Math Formulation

automaton Switch

variables

internal $x, y: \text{Real} := 0, \text{loc}: \{\text{on}, \text{off}\} := \text{off}$

transitions

internal push

pre $x \geq 2$

eff if $\text{loc} = \text{on}$ **then** $x := 0$

else $x, y := 0; \text{loc} := \text{off}$

internal pop

pre $y = 15 \wedge \text{loc} = \text{off}$

eff $x := 0$

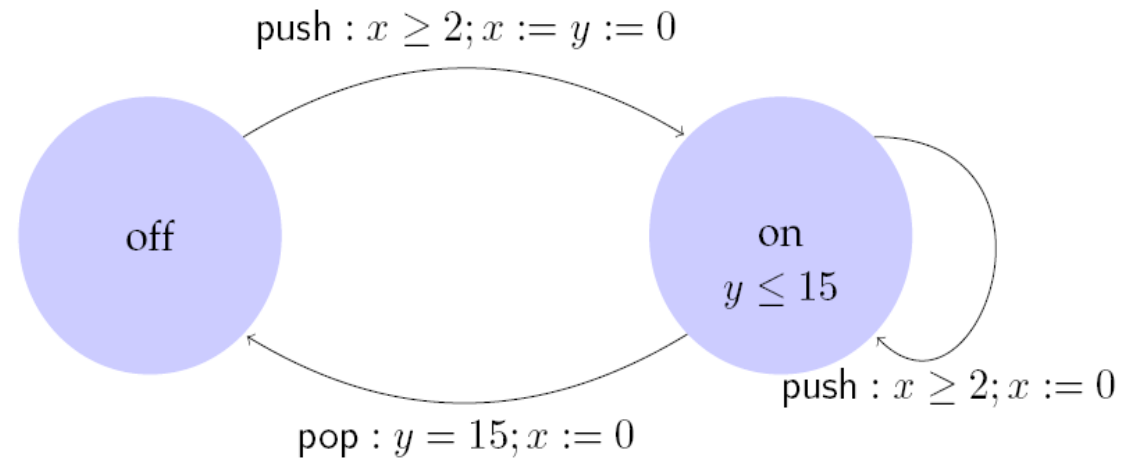
trajectories

invariant $\text{loc} = \text{off} \Rightarrow y \leq 15$

evolve $d(x) = 1; d(y) = 1$

Description

Switch can be turned on whenever at least 2 time units have elapsed since the last turn on. Switches off automatically 15 time units after the last on.

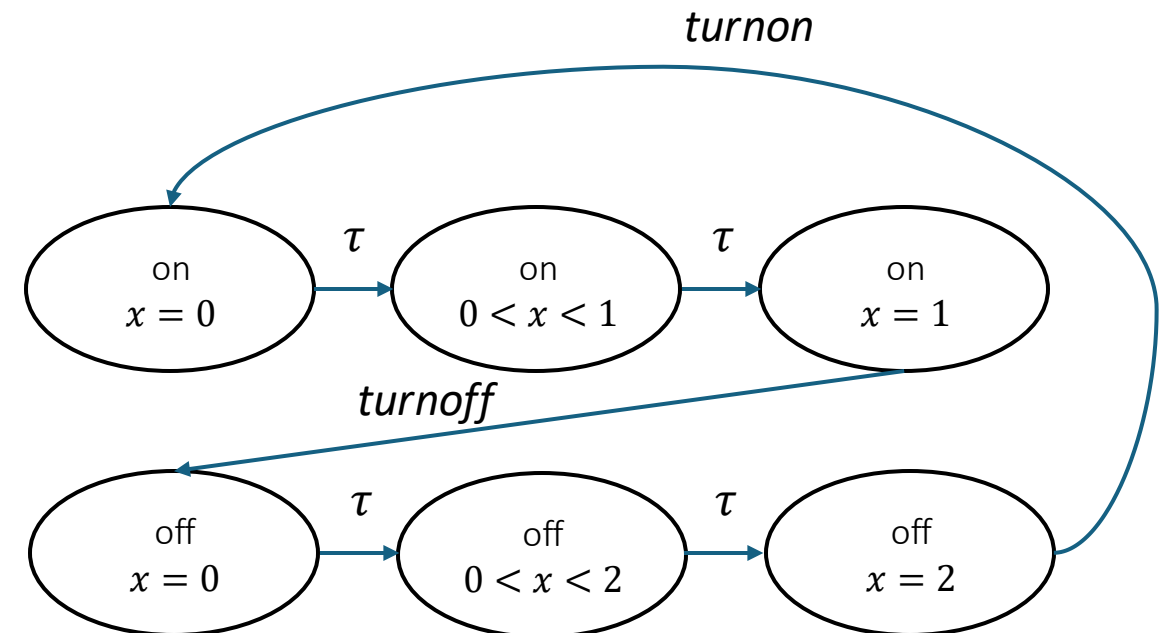
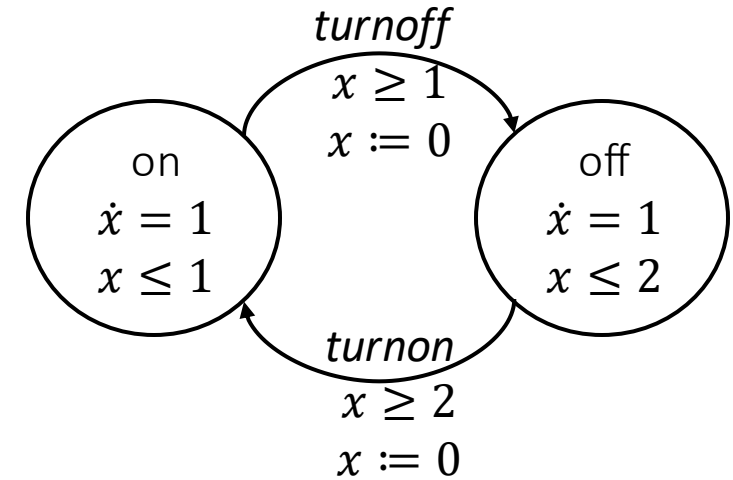


Control State (Location) Reachability Problem

- Given an ITA $\mathcal{A}$, check if a particular (discrete) control state is reachable from the initial states
- Why is control state reachability (CSR) good enough?
- This problem is decidable [Alur Dill]
- Key idea:
 - Construct a finite automaton that is **a time-abstract bisimilar** to the ITA (behaves identically with respect to control state reachability)
 - Check reachability of FSM

An equivalence relation with a finite quotient

- Under what conditions do two states x_1 and x_2 of the automaton $\mathcal{A}$ behave identically with respect to control state reachability (CSR)?
 - When do they satisfy the same set of clock constraints?
 - When would they continue to satisfy the same set of clock constraints?
- If x_1 and x_2 are equivalent, $x_1 \equiv x_2$, from the point of view of reachability then we can use this equivalence relation on the state space Q to define a new automaton $\mathcal{A}'$ with state space $Q' = \text{quotient space } Q/\equiv$ (which is smaller than Q) and perform reachability analysis on $\mathcal{A}'$



An equivalence relation with a finite quotient

- Under what conditions do two states x_1 and x_2 of the automaton $\mathcal{A}$ behave identically with respect to control state reachability (CSR)?
 - When do they satisfy the same set of clock constraints?
 - When would they continue to satisfy the same set of clock constraints?
- $x_1.\text{loc} = x_2.\text{loc}$ and
- x_1 and x_2 satisfy the same set of clock constraints
 - For each clock y $\text{int}(x_1.y) = \text{int}(x_2.y)$ or $\text{int}(x_1.y) \geq c_{\mathcal{A}y}$ and $\text{int}(x_2.y) \geq c_{\mathcal{A}y}$. ($c_{\mathcal{A}y}$ is the maximum clock guard of y)
 - For each clock y with $x_1.y \leq c_{\mathcal{A}y}$, $\text{frac}(x_1.y) = 0$ iff $\text{frac}(x_2.y) = 0$
 - For any two clocks y and z with $x_1.y \leq c_{\mathcal{A}y}$ and $x_1.z \leq c_{\mathcal{A}z}$, $\text{frac}(x_1.y) \leq \text{frac}(x_1.z)$ iff $\text{frac}(x_2.y) \leq \text{frac}(x_2.z)$
- **Lemma.** This is an equivalence relation on $\text{val}(V)$ the states of $\mathcal{A}$
- The partition of $\text{val}(V)$ induced by this relation is called clock regions

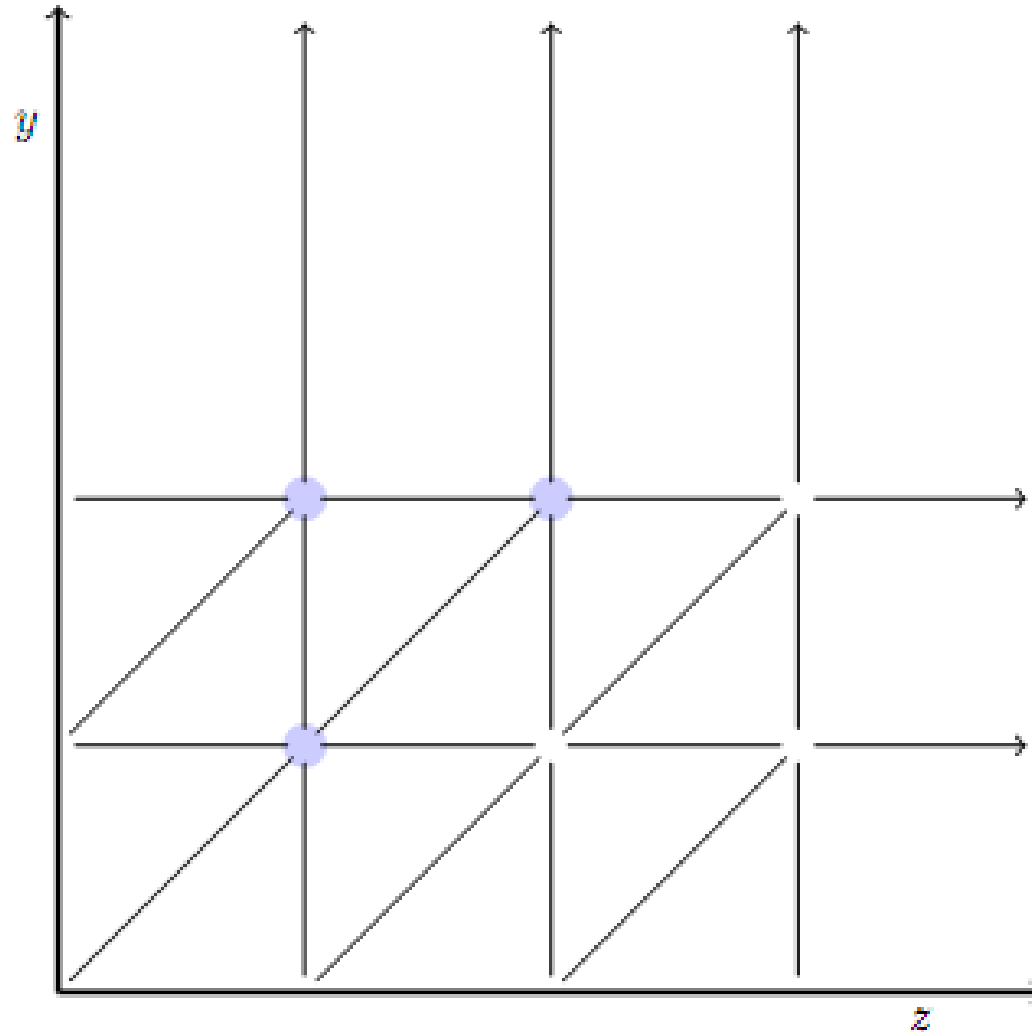
Geometry of clock regions

Example of Two Clocks

$$X = \{y, z\}$$

$$c_{\mathcal{A}y} = 2$$

$$c_{\mathcal{A}z} = 3$$



Complexity

Lemma. The number of clock regions is bounded by $|X|! 2^{|X|} \prod_{z \in X} (2c_{\mathcal{A}_z} + 2)$.

Finite

Region automaton $R(\mathcal{A})$

Given an ITA $\mathcal{A} = \langle V, \Theta, \mathcal{D}, \mathcal{T} \rangle$, we construct the corresponding **Region Automaton** $R(\mathcal{A}) = \langle Q_R, \Theta_R, D_R \rangle$ such that (i) $R(\mathcal{A})$ visits the same set of locations (but does not have timing information) and (ii) $R(\mathcal{A})$ is finite state machine.

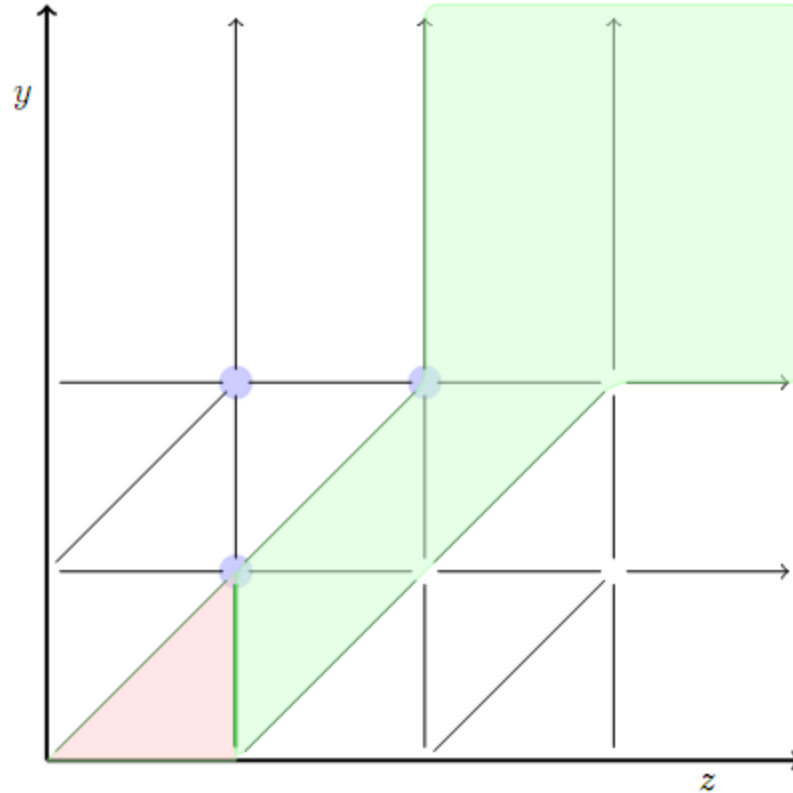
- ITA (clock constants) defines a set of clock regions, say $C_{\mathcal{A}}$. The set of states $Q_R = C_{\mathcal{A}} \times L$
- $Q_0 \subseteq Q$ is the set of states contain initial set Θ of $\mathcal{A}$
- D : We add the transitions between Q (regions)
 - **Time successors**: Consider two clock regions γ and γ' , we say that γ' is a time successor of γ if there exists a trajectory of ITA starting from γ that ends in γ'
 - **Discrete transitions**: Same as the ITA

Theorem. A location of ITA $\mathcal{A}$ is reachable iff it is also reachable in $R(\mathcal{A})$.

(we say that $R(\mathcal{A})$ is *time abstract bisimilar* to $\mathcal{A}$)

Proof. For each execution of $\mathcal{A}$ inductively construct the corresponding execution of $R(\mathcal{A})$ and vice versa.

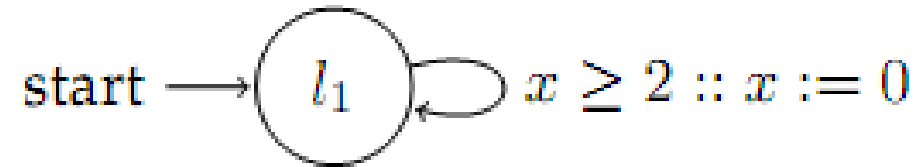
Time successors



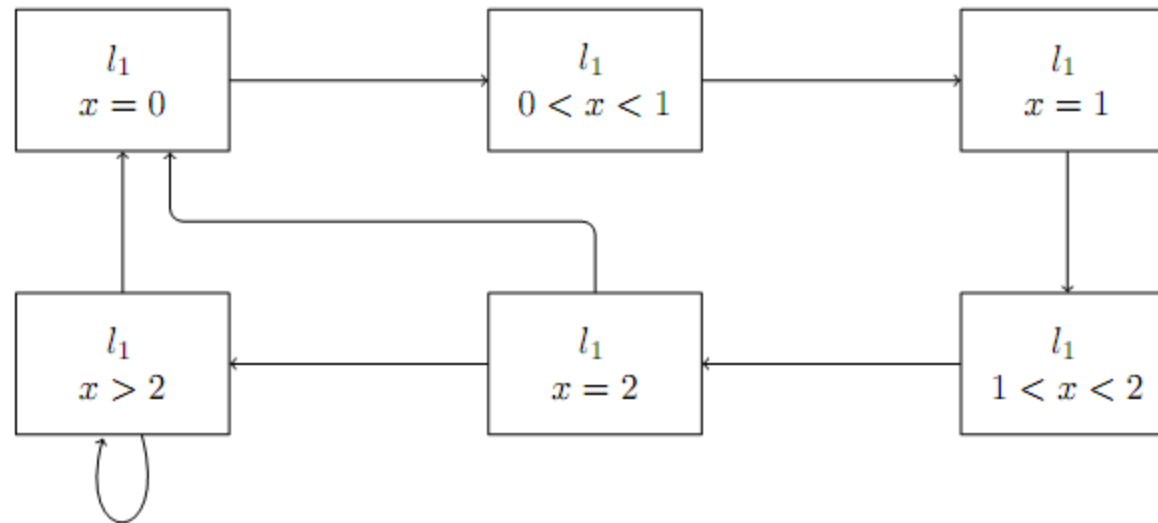
The clock regions in blue are time successors of the clock region in red.

Example 1: Region Automata

ITA

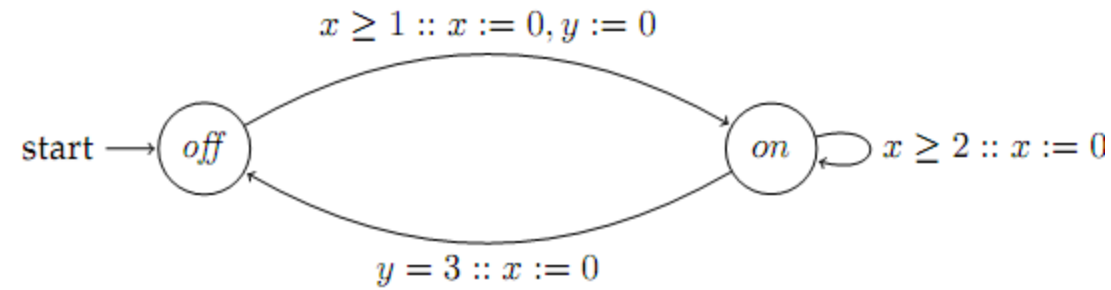


Corresponding FA

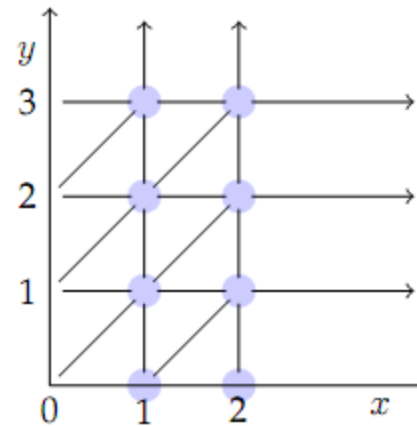


Example 2

ITA



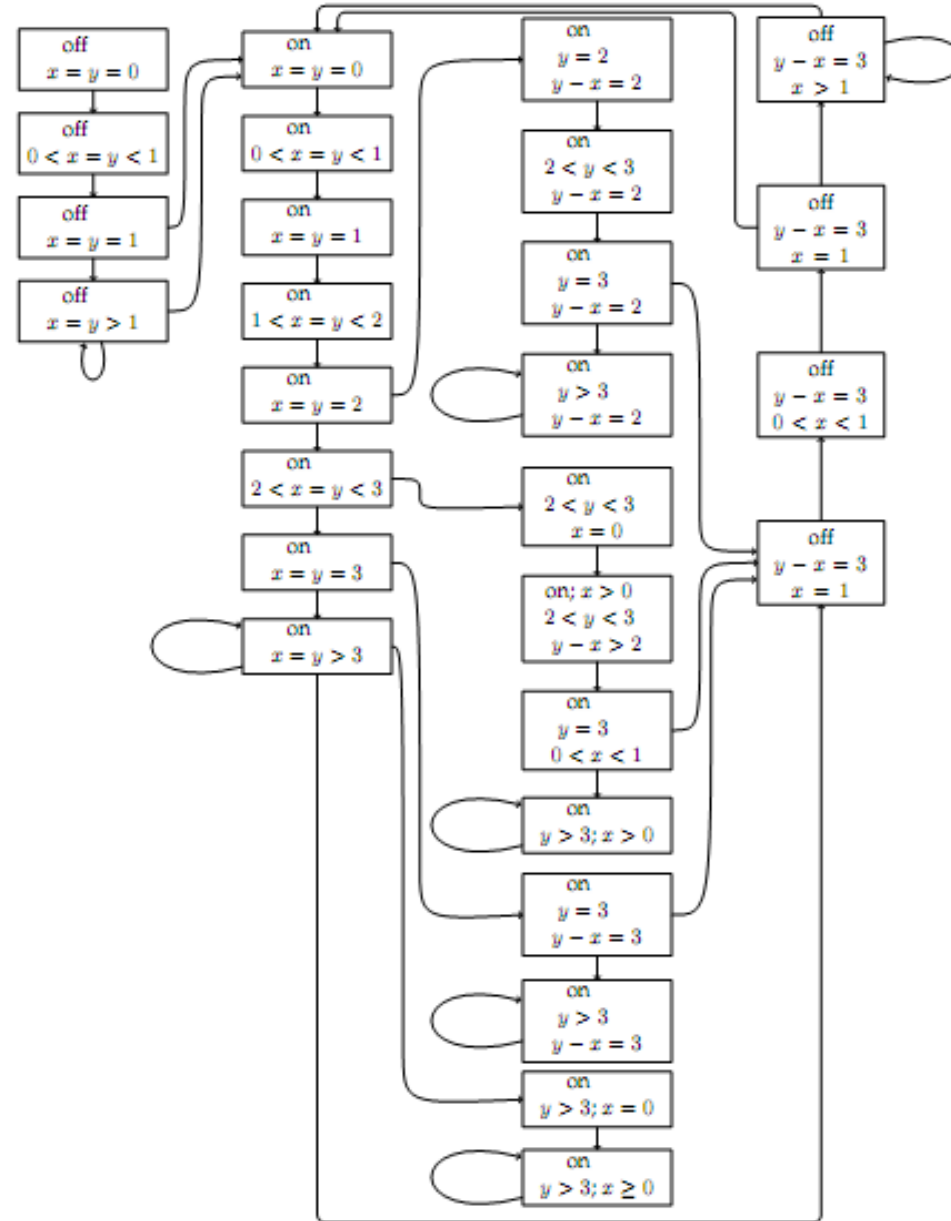
Clock
Regions



Corresponding FA

$$|X|! \cdot 2^{|X|} \prod_{z \in X} (2c_{\mathcal{A}z} + 2)$$

Drastically increasing with
the number of clocks



Summary

- Big idea: If two states behave similarly---simulate each other---then a verification algorithm need not keep track of them separately
 - Related to notion of **indistinguishability**
- Big savings in memory and computation
- Many generalizations
 - Simulation (one way containment) as opposed to bi-simulation
 - Simulation-like relations for stability (not just reachability) and other kinds of requirements
 - Computing simulations offline vs online

Special Classes of Hybrid Automata

- Timed Automata ←
- Rational time automata
- Multirate automata
- Rectangular Initialized HA
- Rectangular HA
- Linear HA
- Nonlinear HA

Clocks and **Rational** Clock Constraints

- A **clock variable** x is a continuous (analog) variable of type real such that along any trajectory τ of x , for all $t \in \tau.\text{dom}$, $(\tau \downarrow x)(t) = t$.
- For a set X of clock variables, the set $\Phi(X)$ of **rational clock constraints** are expressions defined by the syntax:
$$g ::= x \leq q \mid x \geq q \mid \neg g \mid g_1 \wedge g_2$$

where $x \in X$ and $q \in \mathbb{Q}$
- Examples: $x = 10.125$; $x \in [2.99, 5)$; true are valid rational clock constraints
- Semantics of clock constraints $[g]$

Step 1. Rational Timed Automata

- **Definition.** A *rational timed automaton* is a HA $\mathcal{A} = \langle V, \Theta, A, \mathcal{D}, \mathcal{T} \rangle$ where
 - $V = X \cup \{loc\}$, where X is a set of n clocks and l is a discrete state variable of finite type $\mathbb{L}$
 - A is a finite set
 - $\mathcal{D}$ is a set of transitions such that
 - The guards are described by **rational** clock constraints $\Phi(X)$
 - $\langle x, l \rangle - a \rightarrow \langle x', l' \rangle$ implies either $x' = x$ or $x = 0$
 - $\mathcal{T}$ set of clock trajectories for the clock variables in X

Example: Rational Light switch

Switch can be turned on whenever at least 2.25 time units have elapsed since the last turn off or on. Switches off automatically 15.5 time units after the last on.

automaton Switch

internal push; pop

variables

internal $x, y: \text{Real} := 0$, $\text{loc}:\{\text{on}, \text{off}\} := \text{off}$

transitions

push

pre $x \geq 2.25$

eff if $\text{loc} = \text{on}$ then $y := 0$ fi; $x := 0$; $\text{loc} := \text{off}$

pop

pre $y = 15.5 \wedge \text{loc} = \text{off}$

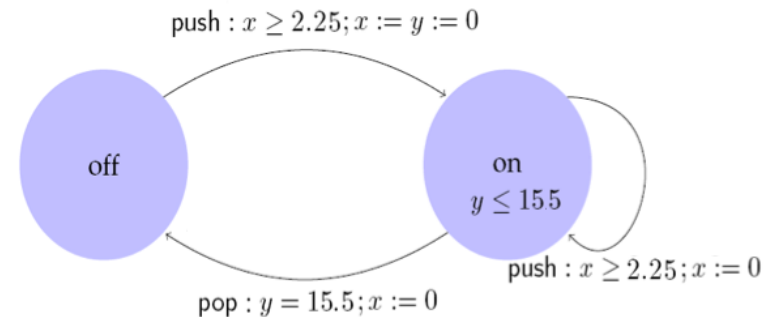
eff $x := 0$

trajectories

invariant $\text{loc} = \text{on} \vee \text{loc} = \text{off}$

stop when $y = 15.5 \wedge \text{loc} = \text{off}$

evolve $d(x) = 1$; $d(y) = 1$



Control State (Location) Reachability Problem

- Given an RTA, check if a particular location is reachable from the initial states
- Is problem decidable?
- Yes
- Key idea:
 - Construct a ITA that is time-abstract bisimilar to the given RTA
 - Check CSR for ITA

Construction of ITA from RTA

- Multiply all rational constants by a factor q that make them integral
- Make $d(x) = q$ for all the clocks
- RTA Switch is bisimilar to ITA lswitch
- Simulation relation R is given by
- $(u, s) \in R$ iff $u.x = 4 s.x$ and $u.y = 4 s.y$

automaton lSwitch

internal push; pop

variables

internal $x, y: \text{Real} := 0, \text{loc}: \{\text{on}, \text{off}\} := \text{off}$

transitions

push

pre $x \geq 9$

eff if $\text{loc} = \text{on}$ **then** $y := 0$ **fi**; $x := 0; \text{loc} := \text{off}$

pop

pre $y = 62 \wedge \text{loc} = \text{off}$

eff $x := 0$

trajectories

invariant $\text{loc} = \text{on} \vee \text{loc} = \text{off}$

stop when $y = 62 \wedge \text{loc} = \text{off}$

evolve $d(x) = 4; d(y) = 4$

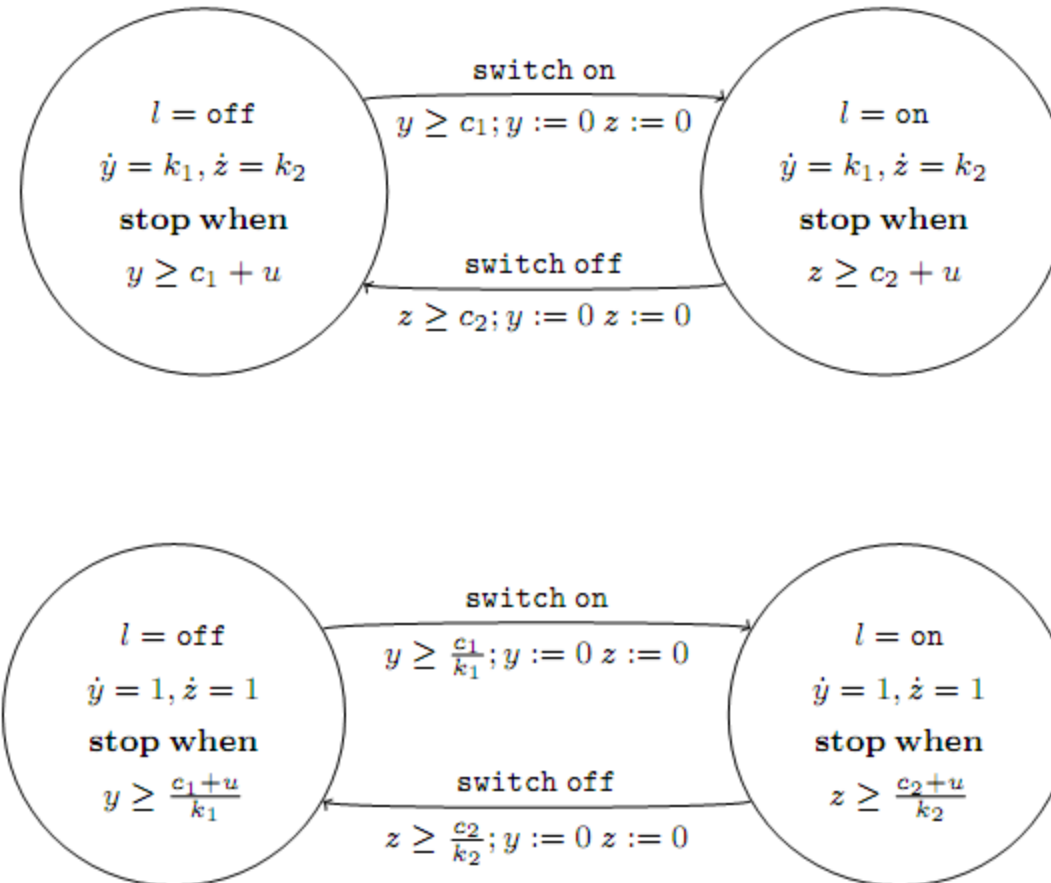
Step 2. Multi-Rate Automaton

- **Definition.** A **multirate automaton** is $\mathcal{A} = \langle V, Q, \Theta, A, \mathcal{D}, \mathcal{T} \rangle$ where
 - $V = X \cup \{loc\}$, where X is a set of n **continuous variables** and loc is a discrete state variable of finite type $\mathbb{L}$
 - A is a finite set of actions
 - $\mathcal{D}$ is a set of transitions such that
 - The guards are described by **rational** clock constraints $\Phi(X)$
 - $\langle x, l \rangle - a \rightarrow \langle x', l' \rangle$ implies either $x' = c$ or $x' = x$
 - $\mathcal{T}$ set of trajectories such that
 - for each variable $x \in X \exists k$ such that $\tau \in \mathcal{T}, t \in \tau.dom$
$$\tau(t).x = \tau(0).x + k t$$

Control State (Location) Reachability Problem

- Given an MRA, check if a particular location is reachable from the initial states
- Is problem is decidable? Yes
- Key idea:
 - Construct a RTA that is bisimilar to the given MRA

Example: Multi-rate to rational TA



Step 3. Rectangular HA

Definition. An **rectangular hybrid automaton (RHA)** is a HA $\mathcal{A} = \langle V, A, \mathcal{T}, \mathcal{D} \rangle$ where

- $V = X \cup \{loc\}$, where X is a set of n **continuous variables** and loc is a discrete state variable of finite type $\mathbb{L}$
- A is a finite set
- $\mathcal{T} = \bigcup_{\ell} \mathcal{T}_{\ell}$ set of trajectories for X
 - For each $\tau \in \mathcal{T}_{\ell}, x \in X$ either (i) $d(x) = k_{\ell}$ or (ii) $d(x) \in [k_{\ell_1}, k_{\ell_2}]$
 - Equivalently, (i) $\tau(t)[x] = \tau(0)[x] + k_{\ell}t$
(ii) $\tau(0)[x] + k_{\ell_1}t \leq \tau(t)[x] \leq \tau(0)[x] + k_{\ell_2}t$
- $\mathcal{D}$ is a set of transitions such that
 - Guards are described by **rational** clock constraints
 - $\langle x, l \rangle \rightarrow_a \langle x', l' \rangle$ implies $x' = x$ or $x' \in [c_1, c_2]$

CSR Decidable for RHA?

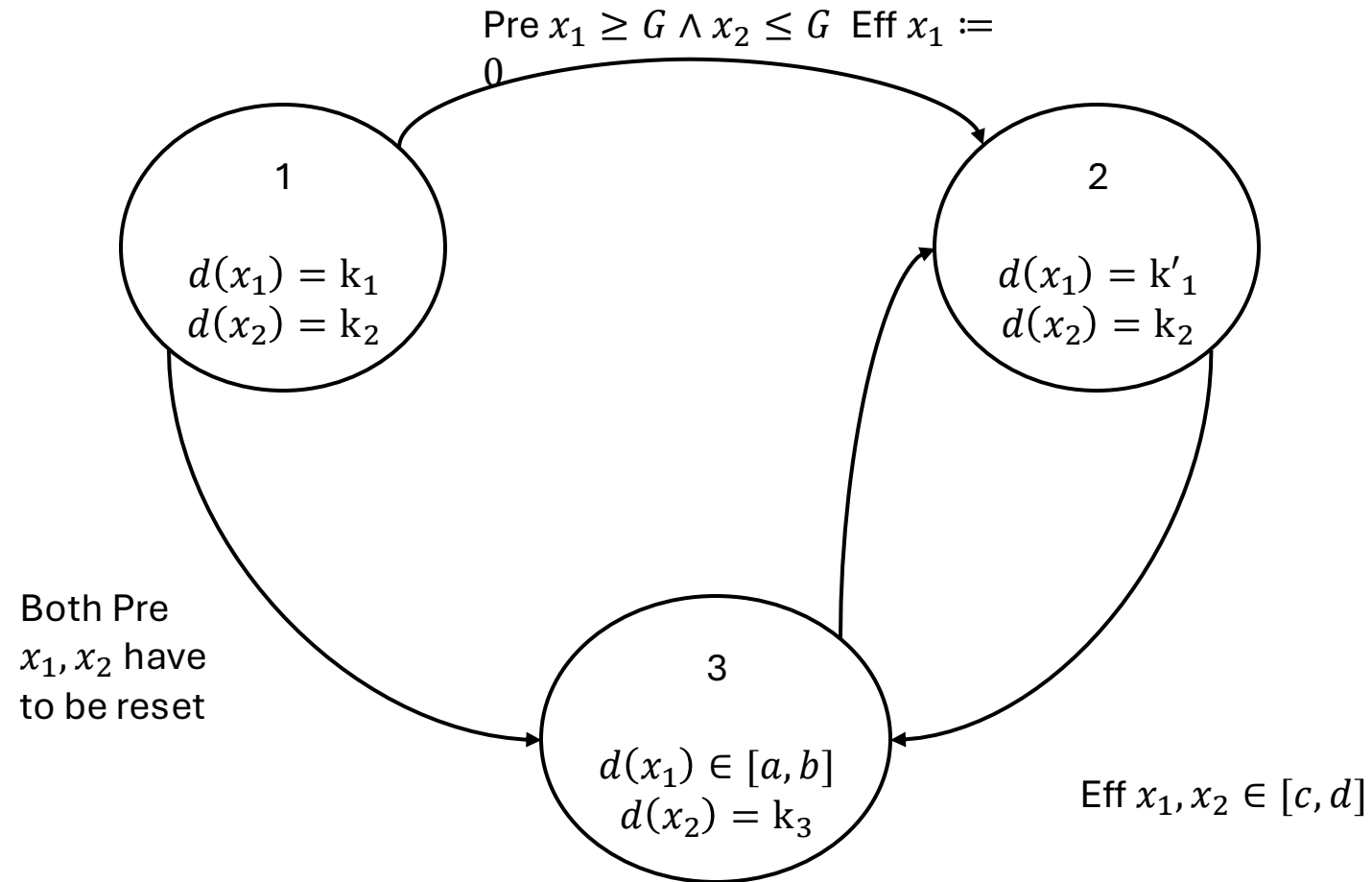
- Given an RHA, check if a particular location is reachable from the initial states?
- Is this problem decidable? **No**
 - **[Henz95]** Thomas Henzinger, Peter Kopke, Anuj Puri, and Pravin Varaiya. [What's Decidable About Hybrid Automata?. Journal of Computer and System Sciences, pages 373–382. ACM Press, 1995.](#)
 - CSR for RHA reduction to Halting problem for 2 counter machines
 - Halting problem for 2CM known to be undecidable
 - Reduction in next lecture

Step 4. Initialized Rectangular HA

Definition. An *initialized rectangular hybrid automaton (IRHA)* is a RHA $\mathcal{A}$ where

- $V = X \cup \{loc\}$, where X is a set of n continuous variables and $\{loc\}$ is a discrete state variable of finite type $\mathbb{L}$
- A is a finite set
- $\mathcal{T} = \bigcup_{\ell} \mathcal{T}_{\ell}$ set of trajectories for X
 - For each $\tau \in \mathcal{T}_{\ell}, x \in X$ either (i) $d(x) = k_{\ell}$ or (ii) $d(x) \in [k_{\ell_1}, k_{\ell_2}]$
 - Equivalently, (i) $\tau(t)[x] = \tau(0)[x] + k_{\ell}t$
(ii) $\tau(0)[x] + k_{\ell_1}t \leq \tau(t)[x] \leq \tau(0)[x] + k_{\ell_2}t$
- $\mathcal{D}$ is a set of transitions such that
 - Guards are described by **rational** clock constraints
 - $\langle x, l \rangle \rightarrow_a \langle x', l' \rangle$ implies if dynamics changes from ℓ to ℓ' then $x' \in [c_1, c_2]$, otherwise $x' = x$

Example: Rectangular Initialized HA



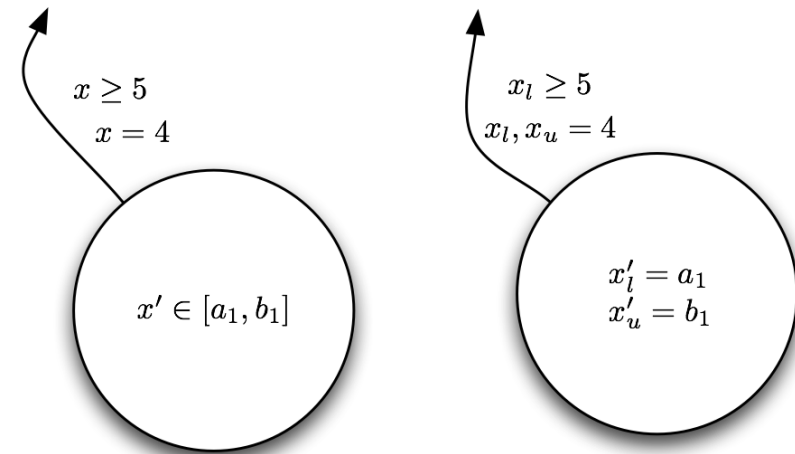
CSR Decidable for IRHA?

- Given an IRHA, check if a particular location is reachable from the initial states
- Is this problem decidable? **Yes**
- Key idea:
 - Construct a $2n$ -dimensional **initialized multi-rate** automaton that is bisimilar to the given IRHA
 - Construct a ITA that is bisimilar to the Singular TA

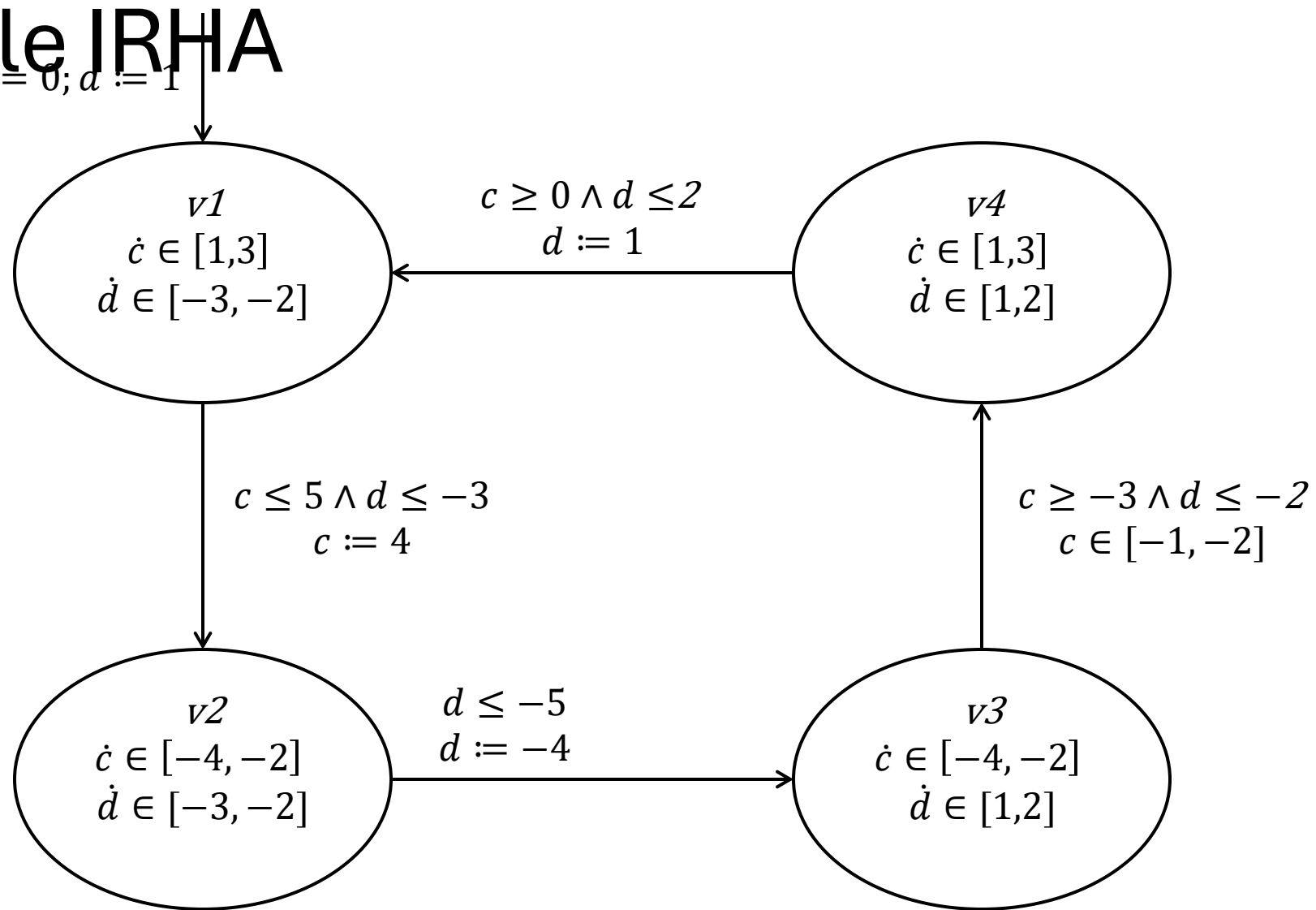
From IRHA to Singular HA conversion

For every variable create two variables---tracking the upper and lower bounds

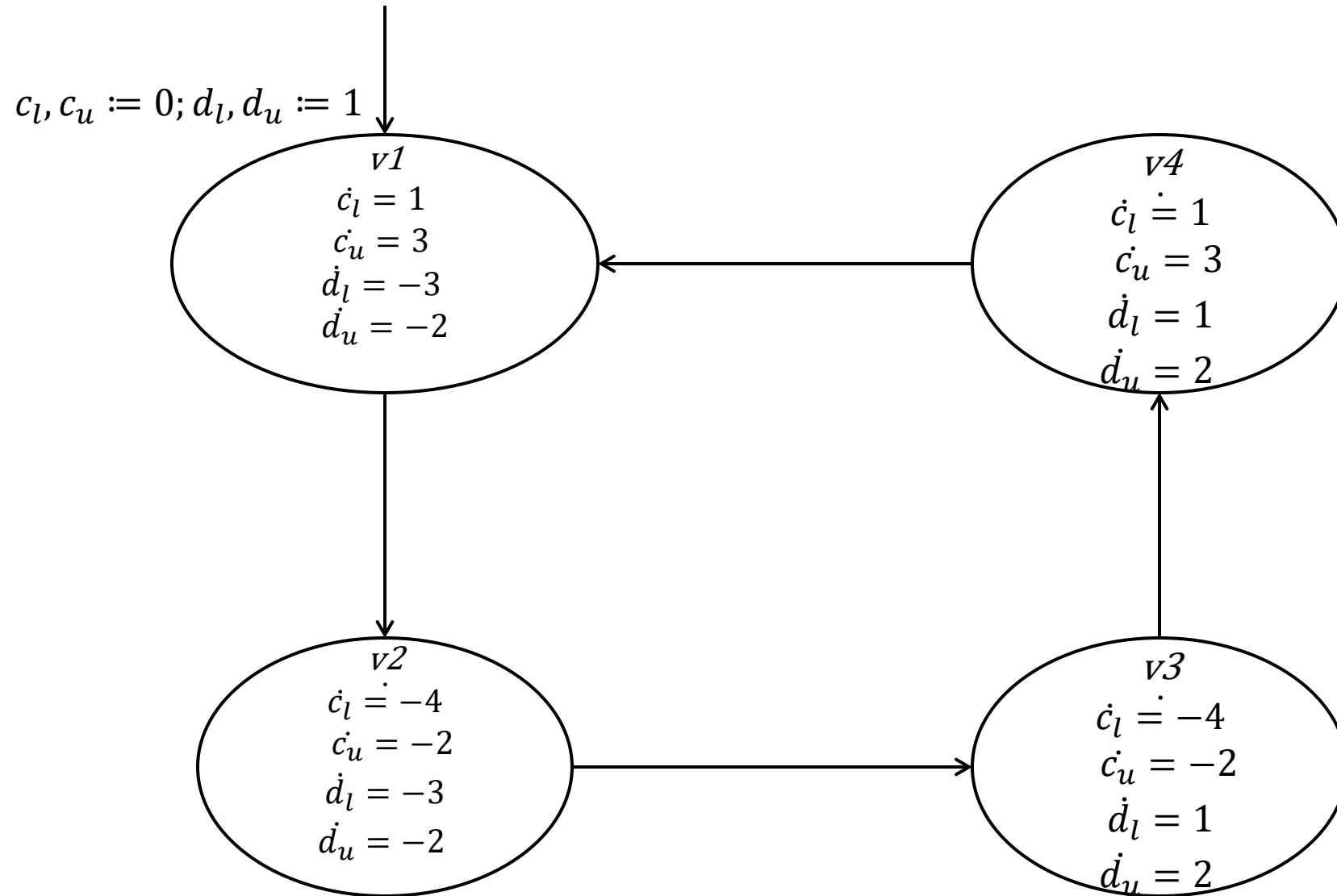
IRHA	MRA
x	$x_\ell ; x_u$
Evolve: $d(x) \in [a_1, b_1]$	Evolve: $d(x_\ell) = a_1; d(x_u) = b_1$
Eff: $x' \in [a_1, b_1]$	Eff: $x_\ell = a_1; x_u = b_1$
$x' = c$	$x_\ell = x_u = c$
Guard: $x \geq 5$	$x_l \geq 5$
	$x_l < 5 \wedge x_u \geq 5 \text{ Eff } x_l = 5$



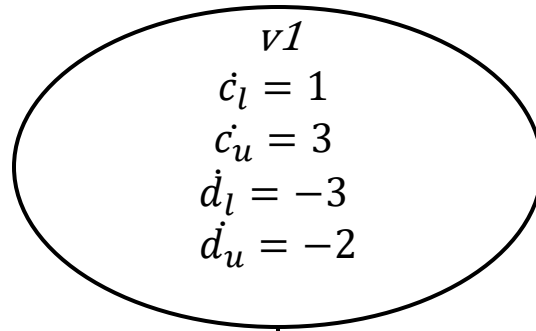
Example IRHA



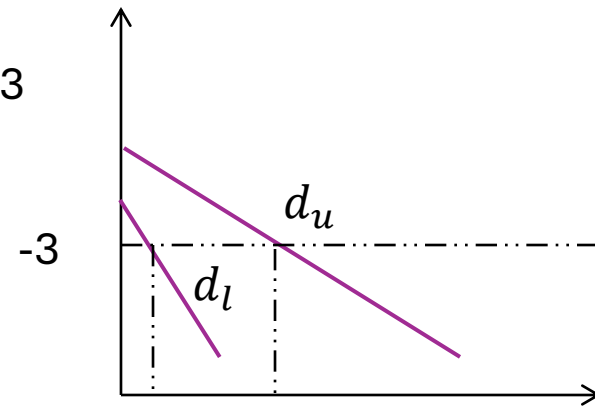
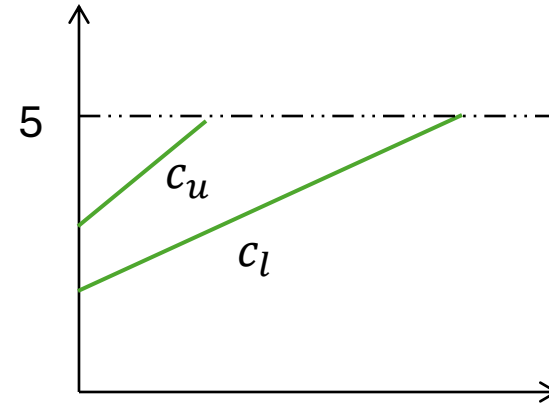
Initialized Singular HA



Transitions

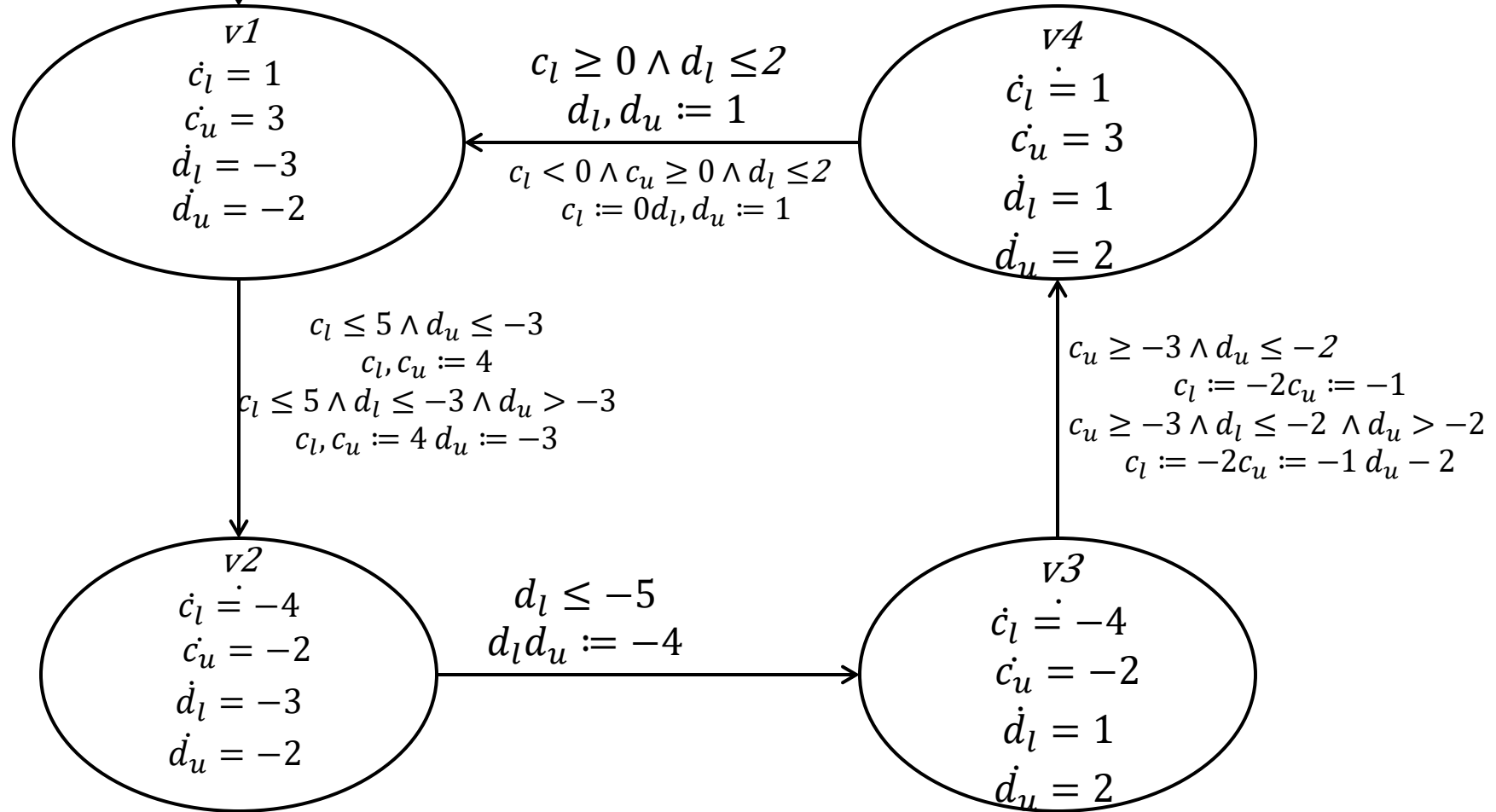


$c_l \leq 5$
 $c_l, c_u := 4$
 $d_u \leq -3$ *no reset*
 $d_u > -3 \wedge d_l \leq -3 \quad d_u := -3$



Initialized Singular HA

$c_l, c_u := 0; d_l, d_u := 1$



Can this be further generalized ?

- For initialized Rectangular HA, control state reachability is decidable
 - Can we drop the initialization restriction?
 - No, problem becomes undecidable (next time)
 - Can we drop the rectangular restriction?
 - No, problem becomes undecidable

Verification in tools

```
Algorithm: BasicReach
2 Input:  $A = \langle V, \Theta, A, D, T \rangle, d > 0$ 
    $Rt, Reach: val(V)$ 
4    $Rt := \Theta;$ 
    $Reach := \emptyset;$ 
6   While ( $Rt \not\subseteq Reach$ )
      $Reach := Reach \cup Rt;$ 
      $Rt := Rt \cup Post_D(Rt);$ 
      $Rt := Post_{T(d)}(Rt);$ 
10 Output:  $Reach$ 
```

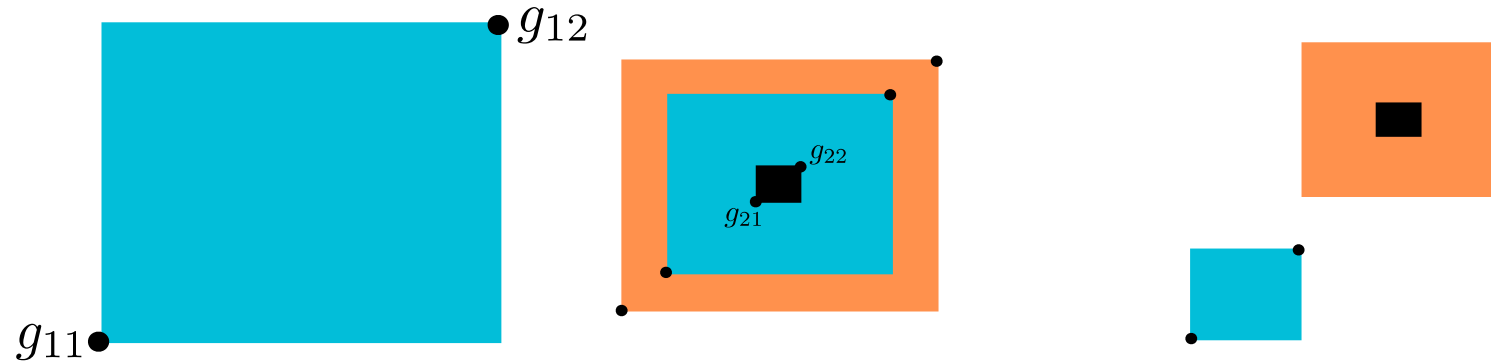
```
Algorithm: PostD
2  $\backslash\backslash$  computes post of all transitions
Input:  $A, D, S_{in}$ 
4    $S_{out} = \emptyset$ 
   For each  $a \in A$ 
6     For each  $\langle g_1, g_2 \rangle \in S_{in}$ 
       If  $\llbracket g_1, g_2 \rrbracket \cap \llbracket g_{a1}, g_{a2} \rrbracket \neq \emptyset$ 
8          $S_{out} := S_{out} \cup \langle g_{a1}, g_{a2} \rangle$ 
Output:  $S_{out}$ 
```

```
1 Algorithm: PostT(d)
    $\backslash\backslash$  computes post of all trajectories
3 Input:  $A, T, S_{in}, d$ 
    $S_{out} = \emptyset$ 
5   For each  $\ell \in L$ 
     For each  $\langle g_1, g_2 \rangle \in S_{in}$ 
7        $P := \cup_{t \leq d} \llbracket g_1, g_2 \rrbracket \oplus \llbracket t g_{\ell 1}, t g_{\ell 2} \rrbracket$ 
        $S_{out} := S_{out} \cup Approx(P)$ 
9 Output:  $S_{out}$ 
```

Data structures make reachability go around

- Hyperrectangles
 - $[g_1; g_2] = \{x \in R^n \mid \|x - g_1\|_\infty \leq \|g_2 - g_1\|_\infty\} = \Pi_i[g_{1i}, g_{2i}]$
- Polyhedra
- Zonotopes [\[Girard 2005\]](#)
- Ellipsoids [\[Kurzhan et al. 2001\]](#)
- Support functions [\[Guernic et al. 2009\]](#)
- Generalized star set [\[Duggirala and Viswanathan 2018\]](#)

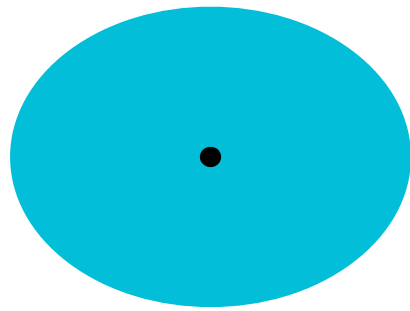
Data structures: rectangles and ellipsoids



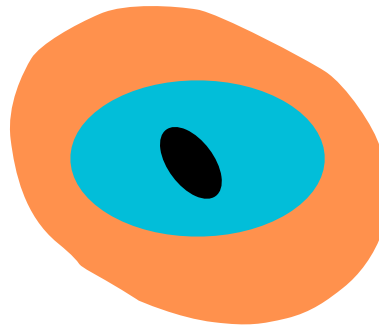
$$[[g_{11}, g_{12}]]$$

$$[[g_{11}, g_{12}]] \oplus [[g_{21}, g_{22}]] \\ = [[g_{11} + g_{12}, g_{21} + g_{22}]]$$

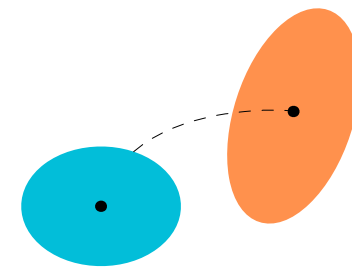
$$[[g_{11}, g_{12}]] \oplus [[t.g_1, t.g_2]]$$



$$[[c_1, Q]]$$

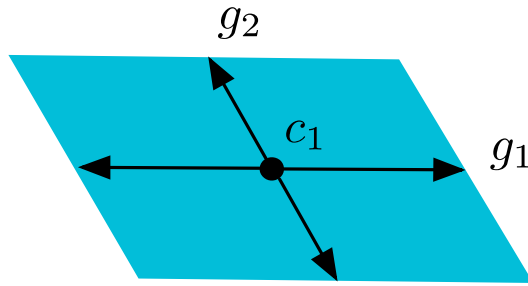


$$[[c_1, Q_1]] \oplus [[c_2, Q_2]] \neq [[c_3, Q_3]]$$

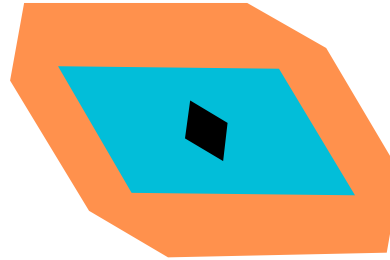


$$[[Ac_1, AQA^T]]$$

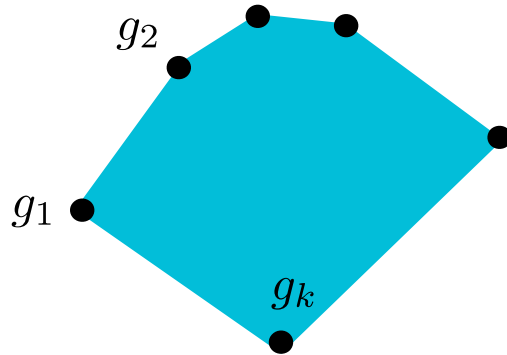
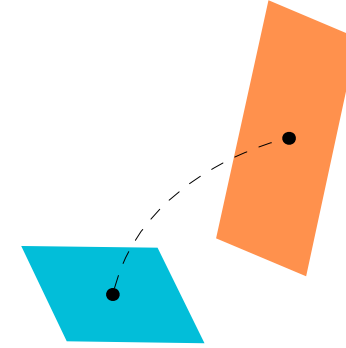
Zonotopes and polytopes



$$[[c_1, \langle g_1, g_2 \rangle]]$$

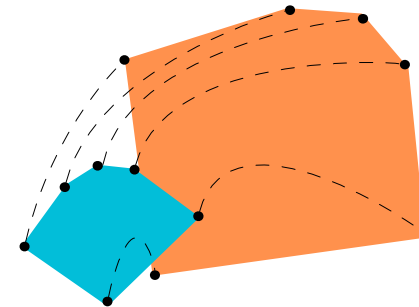
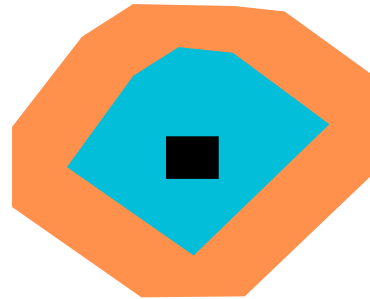


$$[[c_1, \langle g_1, g_2 \rangle]] \oplus [[c_2, \langle g'_1, g'_2 \rangle]] \quad [[Ac_1, \langle Ag_1, Ag_2 \rangle]] \\ = [[c_1 + c_2, \langle g_1, g'_1, g_2, g'_2 \rangle]]$$



$$[[A, b]]$$

$$[[g_1, \dots, g_k]]$$



$$[[\xi(g_1, t), \dots, \xi(g_k, t)]]$$

Summary

- ITA: Restricted class of hybrid automata
 - Clocks, integer constraints
 - No clock comparison, linear
- Control state reachability with Alur-Dill's algorithm (region automaton construction)
- Rational coefficients; multirate Automata
- Initialized Rectangular Hybrid Automata
- HyTech, PHAVer use polyhedral reachability computations